

LINUX TO AZURE

Automated System Log Backup to Azure Blob Storage

Ubuntu CLI → Azure CLI → Blob Storage → Cron Automation

Project Title	Linux to Azure: Automated System Log Backup to Blob Storage
Date	September 16, 2026
Analyst	Bryan Sykes
Source Machine	sykes-ubuntu (ASUS X550ZA — Ubuntu Desktop 26.04 LTS)
Azure Resource Group	LinuxLogsRG (East US)
Azure Storage Account	sykeslogstorage
Blob Container	syslogs
Automation	Cron job: daily at midnight
Coverage	Azure CLI Setup, Resource Provisioning, Bash Scripting, RBAC, Cron Automation, Troubleshooting

Analyst: Bryan Sykes | Home Lab Portfolio Project

1. Project Overview

This project builds an automated pipeline that compresses Ubuntu system logs and uploads them to Microsoft Azure Blob Storage on a daily cron schedule entirely from the command line over SSH. The pipeline demonstrates the intersection of three skill domains that are directly applicable to federal IT and cloud operations roles: Linux system administration, Azure cloud resource management, and bash scripting automation.

The project was built on sykes-ubuntu, a dedicated physical Ubuntu Desktop 26.04 LTS machine that is administered entirely via SSH from a primary Windows desktop workstation. No GUI tools were used at any stage of this project. Every Azure resource was provisioned, configured, and tested through the Azure CLI running in an SSH terminal session.

This project was inspired by a Google AI-generated cloud project guide covering hardware-to-cloud log pipelines with Azure Blob Storage and Azure Monitor. The implementation was adapted for the existing home lab environment and extended with RBAC configuration and automated scheduling.

1.1 Skills Demonstrated

Skill Domain	Specific Skills Applied
Linux System Administration	SSH remote administration, bash scripting, file compression (gzip), cron job scheduling, chmod permissions
Azure Cloud (AZ-900 aligned)	Azure CLI authentication, resource groups, storage accounts, blob containers, RBAC role assignment
Security	Role-based access control (RBAC), Storage Blob Data Contributor assignment, encrypted transfer (HTTPS), server-side encryption confirmed
Automation	Cron job scheduling, bash script with dynamic date variables, log output redirection, temp file cleanup
Troubleshooting	Azure AD tenant issues, RBAC permission errors, authentication failures, Yahoo/Microsoft account federation

2. Environment

2.1 Source Machine

Component	Detail
Hostname	sykes-ubuntu
Hardware	ASUS X550ZA - AMD A8-7100, 8GB DDR3, 256GB SATA SSD
Operating System	Ubuntu Desktop 26.04 LTS (Resolute Raccoon)
Access Method	SSH from primary Windows desktop (192.168.1.162) — no direct physical interaction
Log Source	/var/log/syslog

2.2 Azure Resources Provisioned

Resource	Name	Configuration
Resource Group	LinuxLogsRG	East US region
Storage Account	sykeslogstorage	Standard LRS, StorageV2, HTTPS only, server-side encryption enabled
Blob Container	syslogs	Private access (no public blob access)
RBAC Role	Storage Blob Data Contributor	Assigned to signed-in user at storage account scope

3. Azure CLI Installation and Authentication

3.1 Installing Azure CLI on Ubuntu

Azure CLI was installed on sykes-ubuntu via SSH using Microsoft's official Debian/Ubuntu install script. curl was not installed by default on the Ubuntu 26.04 machine and had to be installed first:

```
sudo apt install curl -y
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az --version      # confirmed: up to date
```

3.2 Authentication: Challenges and Resolution

Authentication to Azure via CLI involved several challenges due to the account configuration. The Azure account was created under a Yahoo email address federated to a Microsoft account. Microsoft's security defaults for this tenant type caused repeated authentication failures.

```

bryansykes@sykes-ubuntu:~$ az login --use-device-code
To sign in, use a web browser to open the page https://login.microsoft.com/device and enter the code E5YC7FWP7 to authenticate.

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Azure subscription 1  ed6e958b-2f8b-475b-a5e4-97595e1b4beb  Default Directory

The default is marked with an *; the default tenant is 'Default Directory' and subscription is 'Azure subscription 1' (ed6e958b-2f8b-475b-a5e4-97595e1b4beb).

Select a subscription and tenant (Type a number or Enter for no changes): |

```

Figure 1 — Successful az login showing Azure subscription 1 enabled and set as default

Issue	Cause	Resolution
No subscriptions found after login	First az login attempt interrupted with Ctrl+C before completion, authentication never fully processed	Ran az login again and waited for full completion
Authentication failed — AADSTS530035	Microsoft security defaults blocked device code flow for Yahoo-federated Microsoft accounts	Disabled security defaults in Microsoft Entra ID (Azure AD) → Properties → Manage Security Defaults

Full Azure portal logout	Multiple failed CLI login attempts flagged as suspicious by Microsoft and locked the account entirely	Recovered via account.live.com/password/reset password reset restored portal and CLI access
Subscription visible but CLI returned "no subscriptions"	Tenant/subscription mismatch. CLI defaulted to wrong tenant context	Used az login --tenant <tenant-id> --use-device-code to specify the correct Default Directory tenant

```

The default is marked with an *; the default tenant is 'Default Directory' and subscription is 'Azure subscription 1' (ed6e958b-2f8b-475b-a5e4-97595e1b4beb).

Select a subscription and tenant (Type a number or Enter for no changes):

Tenant: Default Directory
Subscription: Azure subscription 1 (ed6e958b-2f8b-475b-a5e4-97595e1b4beb)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.

mrbryansykes@sykes-ubuntu:~$ az account list --output table
Name                        CloudName      SubscriptionId                               TenantId                               State
-----
Azure subscription 1      AzureCloud    ed6e958b-2f8b-475b-a5e4-97595e1b4beb      525b4515-7b98-48d4-914a-2b7a1ad9bbb8  Enabled
True
mrbryansykes@sykes-ubuntu:~$ |

```

Figure 2 — Azure subscription 1 confirmed in CLI after tenant-specific login

Final successful authentication command:

```
az login --tenant 525b4515-7b98-48d4-914a-2b7a1ad9bbb8 --use-device-code
```

```

Your CLI is up-to-date.
mrbryansykes@sykes-ubuntu:~$ az login
To sign in, use a web browser to open the page https://login.microsoft.com/device and enter the code FA8SJ9JSW to authenticate.
mrbryansykes@sykes-ubuntu:~$ az account list --output table
Please run "az login" to access your accounts.

mrbryansykes@sykes-ubuntu:~$ az login
To sign in, use a web browser to open the page https://login.microsoft.com/device and enter the code AXGQ66C5Z to authenticate.

Retrieving tenants and subscriptions for the selection...
Authentication failed against tenant 525b4515-7b98-48d4-914a-2b7a1ad9bbb8 'Default Directory': AADSTS530035: Access has been blocked by security defaults. Trace ID: 7409836c-0a1e-4266-ade9-0b7215045100 Correlation ID: 85baa497-a435-4a9f-013-1589f7b7ffcd Timestamp: 2026-09-16 20:44:43Z
If you need to access subscriptions in the following tenants, please use 'az login --tenant TENANT_ID'.
525b4515-7b98-48d4-914a-2b7a1ad9bbb8 'Default Directory'
No subscriptions found for bballer7299@yahoo.com.
mrbryansykes@sykes-ubuntu:~$ |

```

Figure 3 — Initial authentication failure: no subscriptions found and tenant security defaults blocking access

4. Azure Resource Provisioning

4.1 Resource Group

A dedicated resource group was created to contain all project resources, following Azure best practice of grouping related resources for lifecycle management:

```
az group create --name LinuxLogsRG --location eastus
```

```

bryansykes@sykes-ubuntu:~$ az group create --name LinuxLogsRG --location eastus
az storage account create --name sykeslogstorage --resource-group LinuxLogsRG --location eastus --sku Standard_LRS
{
  "id": "/subscriptions/ed6e958b-2f8b-475b-a5e4-97595e1b4beb/resourceGroups/LinuxLogsRG",
  "location": "eastus",
  "managedBy": null,
  "name": "LinuxLogsRG",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
{
  "accessTier": "Hot",
  "accountMigrationInProgress": null,
  "allowBlobPublicAccess": false,
  "allowCrossTenantReplication": false,
  "allowSharedKeyAccess": null,
  "allowSharedKeyAccessForServices": null,
  "allowedCopyScope": null,
  "azureFilesIdentityBasedAuthentication": null,
  "blobRestoreStatus": null,
  "creationTime": "2026-09-16T21:00:27.889093+00:00",
  "customDomain": null,
  "dataCollaborationPolicyProperties": null,
  "defaultOAuthAuthentication": null,
  "dnsEndpointType": null,
  "dualStackEndpointPreference": null,
  "enableExtendedGroups": null,
  "enableHttpsTrafficOnly": true,
  "enableNfsV3": null,
  "encryption": {
    "encryptionIdentity": null,
    "keySource": "Microsoft.Storage",
    "keyVaultProperties": null,
    "requireInfrastructureEncryption": null,
    "services": {
      "blob": {
        "enabled": true,
        "keyType": "Account",
        "lastEnabledTime": "2026-09-16T21:00:28.346855+00:00"
      },
      "file": {
        "enabled": true,
        "keyType": "Account",
        "lastEnabledTime": "2026-09-16T21:00:28.346855+00:00"
      },
      "queue": null,
      "table": null
    }
  },
  "extendedLocation": null,
  "failoverInProgress": null,
  "geoPriorityReplicationStatus": null,
  "geoReplicationStats": null,
  "id": "/subscriptions/ed6e958b-2f8b-475b-a5e4-97595e1b4beb/resourceGroups/LinuxLogsRG/providers/Microsoft.Storage/storageAccounts/sykeslogstorage",
  "identity": null,
  "immutableStorageWithVersioning": null,
  "isHttpsEnabled": null,

```

Figure 4 — Resource group LinuxLogsRG created successfully — provisioningState: Succeeded

4.2 Storage Account

A Standard LRS (Locally Redundant Storage) storage account was provisioned within the resource group. Standard LRS was selected as the appropriate tier for a home lab log backup use case, cost effective with adequate durability for non-critical data:

```

az storage account create --name sykeslogstorage --resource-group LinuxLogsRG --
location eastus --sku Standard_LRS

```

```

},
"keyPolicy": null,
"kind": "StorageV2",
"largeFileSharesState": null,
"lastGeoFailoverTime": null,
"location": "eastus",
"minimumTlsVersion": "TLS1_0",
"name": "sykeslogstorage",
"networkRuleSet": {
  "bypass": "None",
  "defaultAction": "Allow",
  "ipRules": [],
  "ipv6Rules": [],
  "resourceAccessRules": null,
  "virtualNetworkRules": []
},
"placement": null,
"primaryEndpoints": {
  "blob": "https://sykeslogstorage.blob.core.windows.net/",
  "dfs": "https://sykeslogstorage.dfs.core.windows.net/",
  "file": "https://sykeslogstorage.file.core.windows.net/",
  "internetEndpoints": null,
  "ipv6Endpoints": null,
  "microsoftEndpoints": null,
  "queue": "https://sykeslogstorage.queue.core.windows.net/",
  "table": "https://sykeslogstorage.table.core.windows.net/",
  "web": "https://sykeslogstorage.z13.web.core.windows.net/"
},
"primaryLocation": "eastus",
"privateEndpointConnections": [],
"provisioningState": "Succeeded",
"publicNetworkAccess": null,
"resourceGroup": "LinuxLogsRG",
"routingPreference": null,
"sasPolicy": null,
"secondaryEndpoints": null,
"secondaryLocation": null,
"sku": {
  "name": "Standard_LRS",
  "tier": "Standard"
},
"statusOfPrimary": "available",
"statusOfSecondary": null,
"storageAccountSkuConversionStatus": null,
"systemData": null,
"tags": {},
"type": "Microsoft.Storage/storageAccounts",
"zones": null
}
mrbryansykes@sykes-ubuntu:~$

```

Figure 5 — Storage account sykeslogstorage created — StorageV2, Standard LRS, HTTPS-only, server-side encryption enabled

4.3 Blob Container

A blob container named syslogs was created within the storage account to hold the compressed log files. Authentication mode was set to login (Entra ID / RBAC) rather than account key — the more secure approach:

```
az storage container create --name syslogs --account-name sykeslogstorage --auth-mode login
```

Result: { "created": true }

4.4 RBAC Role Assignment

The initial backup script execution failed with a permissions error the authenticated user lacked blob data plane access despite owning the storage account. This is expected behavior in Azure: control plane access (creating resources) and data plane access (reading/writing blobs) are governed by separate RBAC roles.

```

mrbryansykes@sykes-ubuntu:~$ nano ~/backup.sh
mrbryansykes@sykes-ubuntu:~$ chmod +x ~/backup.sh
mrbryansykes@sykes-ubuntu:~$ ~/backup.sh

You do not have the required permissions needed to perform this operation.
Depending on your operation, you may need to be assigned one of the following roles:
  "Storage Blob Data Owner"
  "Storage Blob Data Contributor"
  "Storage Blob Data Reader"
  "Storage Queue Data Contributor"
  "Storage Queue Data Reader"
  "Storage Table Data Contributor"
  "Storage Table Data Reader"

If you want to use the old authentication method and allow querying for the right account key, please use the "--auth-mode" parameter and "key" value.

Backup completed: syslog-2026-09-16.gz uploaded to Azure
mrbryansykes@sykes-ubuntu:~$

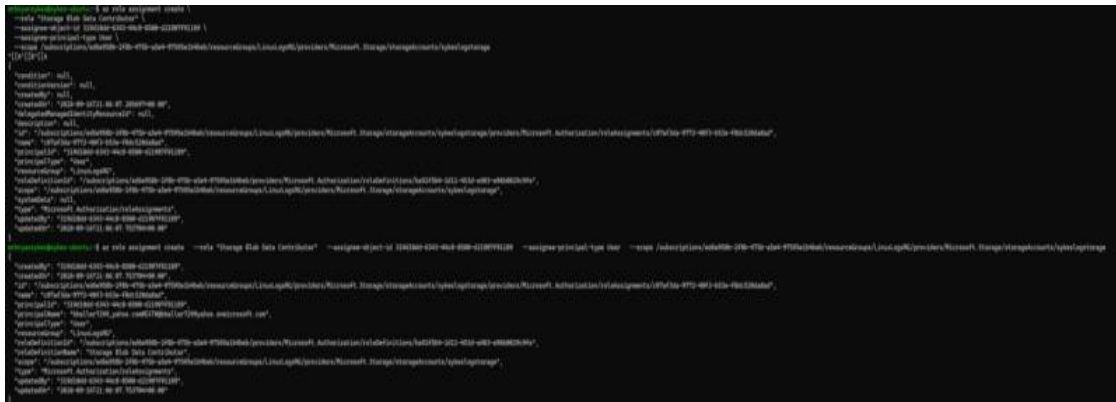
```

Figure 6 — Permissions error on first backup run. Storage Blob Data Contributor role required

The user's Object ID was retrieved and the Storage Blob Data Contributor role was assigned at the storage account scope:

```
az ad signed-in-user show --query id -o tsv
# Returned: 319d10dd-6343-44c8-8500-d21907f91189
```

```
az role assignment create \
  --role "Storage Blob Data Contributor" \
  --assignee-object-id 319d10dd-6343-44c8-8500-d21907f91189 \
  --assignee-principal-type User \
  --scope /subscriptions/ed6e958b-2f8b-475b-a5e4-97595e1b4beb/resourceGroups/LinuxLogsRG/providers/Microsoft.Storage/storageAccounts/sykeslogstorage
```



```
az role assignment create \
  --role "Storage Blob Data Contributor" \
  --assignee-object-id 319d10dd-6343-44c8-8500-d21907f91189 \
  --assignee-principal-type User \
  --scope /subscriptions/ed6e958b-2f8b-475b-a5e4-97595e1b4beb/resourceGroups/LinuxLogsRG/providers/Microsoft.Storage/storageAccounts/sykeslogstorage
{
  "id": "319d10dd-6343-44c8-8500-d21907f91189",
  "roleDefinitionName": "Storage Blob Data Contributor",
  "principalType": "User",
  "principalId": "319d10dd-6343-44c8-8500-d21907f91189",
  "scope": "/subscriptions/ed6e958b-2f8b-475b-a5e4-97595e1b4beb/resourceGroups/LinuxLogsRG/providers/Microsoft.Storage/storageAccounts/sykeslogstorage",
  "status": "Succeeded"
}
```

Figure 7 — Storage Blob Data Contributor role assigned — roleDefinitionName and principalType: User confirmed

Note: The assignee email (bballer7299@yahoo.com) could not be resolved directly in the Azure AD graph due to Yahoo account federation limitations. The Object ID was used instead the correct approach for non-native Microsoft accounts in Azure RBAC assignments.

5. Backup Script

5.1 Script Overview

A bash script was written to handle the complete backup workflow: compress the syslog file with gzip, upload the compressed archive to Azure Blob Storage, and clean up the local temporary file. The script uses a dynamic date variable to create uniquely named archives for each run.

5.2 Script Contents (backup.sh)

```
#!/bin/bash

# Variables
STORAGE_ACCOUNT="sykeslogstorage"
CONTAINER="syslogs"
DATE=$(date +%Y-%m-%d)
BACKUP_FILE="/tmp/syslog-$(DATE).gz"
```

```
# Compress syslog
gzip -c /var/log/syslog > $BACKUP_FILE

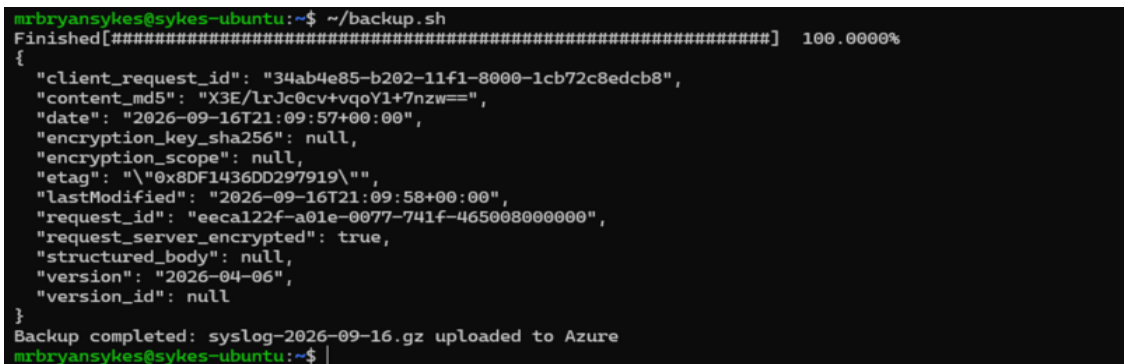
# Upload to Azure Blob Storage
az storage blob upload \
  --account-name $STORAGE_ACCOUNT \
  --container-name $CONTAINER \
  --name "syslog-$DATE.gz" \
  --file $BACKUP_FILE \
  --auth-mode login

# Clean up local temp file
rm $BACKUP_FILE

echo "Backup completed: syslog-$DATE.gz uploaded to Azure"
```

5.3 Script Deployment

- Created script: nano ~/backup.sh
- Made executable: chmod +x ~/backup.sh
- Tested manually: ~/backup.sh



```
mrbryansykes@sykes-ubuntu:~$ ~/backup.sh
Finished[#####] 100.0000%
{
  "client_request_id": "34ab4e85-b202-11f1-8000-1cb72c8edcb8",
  "content_md5": "X3E/lrJc0cv+vqoY1+7nzw==",
  "date": "2026-09-16T21:09:57+00:00",
  "encryption_key_sha256": null,
  "encryption_scope": null,
  "etag": "\"0x8DF1436DD297919\"",
  "lastModified": "2026-09-16T21:09:58+00:00",
  "request_id": "eecal22f-a01e-0077-741f-465008000000",
  "request_server_encrypted": true,
  "structured_body": null,
  "version": "2026-04-06",
  "version_id": null
}
Backup completed: syslog-2026-09-16.gz uploaded to Azure
mrbryansykes@sykes-ubuntu:~$
```

Figure 8 — Successful backup: syslog-2026-09-16.gz uploaded to Azure. 100% progress bar and upload metadata confirmed

Upload confirmation details visible in output:

- content_md5: checksum confirming file integrity
- request_server_encrypted: true, confirming server-side encryption active
- lastModified: 2026-09-16T21:09:58+00:00
- version: 2026-04-06 current Azure Storage API version

6. Cron Job Automation

The backup script was added to the user crontab to run automatically every night at midnight. Output is redirected to a log file for monitoring and troubleshooting:

```
crontab -e
```

Entry added to crontab:

```
0 0 * * * /home/mrbryansykes/backup.sh >> /home/mrbryansykes/backup.log 2>&1
```

Cron Field	Value	Meaning
Minute	0	At minute 0 (top of the hour)
Hour	0	At midnight (00:00)
Day of Month	*	Every day
Month	*	Every month
Day of Week	*	Every day of the week
Output	>> backup.log 2>&1	Appends both stdout and stderr to backup.log for monitoring

The cron entry was verified with `crontab -l` confirming the job is scheduled and active. A new compressed syslog archive named `syslog-YYYY-MM-DD.gz` will be uploaded to the `syslogs` container in Azure Blob Storage every night at midnight automatically.

7. System Hardening & Sleep Prevention

During this session it was discovered that closing the laptop lid caused the machine to suspend, terminating active SSH sessions and dropping network connectivity. Two fixes were applied to permanently prevent this:

7.1 Lid Close Behavior

The systemd login manager was configured to ignore lid close events:

```
sudo nano /etc/systemd/logind.conf
```

Changed: `#HandleLidSwitch=suspend`

To: `HandleLidSwitch=ignore`

```
sudo systemctl restart systemd-logind
```

7.2 Mask All Sleep Targets

To prevent GNOME power settings from overriding the logind configuration, all systemd sleep targets were masked entirely, making sleep impossible at the OS level regardless of any application or user settings:

```
sudo systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
```

```

bryansykes@sykes-ubuntu:~$ sudo systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
[sudo: authenticate] Password:
Created symlink '/etc/systemd/system/sleep.target' → '/dev/null'.
Created symlink '/etc/systemd/system/suspend.target' → '/dev/null'.
Created symlink '/etc/systemd/system/hibernate.target' → '/dev/null'.
Created symlink '/etc/systemd/system/hybrid-sleep.target' → '/dev/null'.
bryansykes@sykes-ubuntu:~$ sudo systemctl status sleep.target
● sleep.target
   Loaded: masked (Reason: Unit sleep.target is masked.)
   Active: inactive (dead) since Wed 2026-09-16 16:36:21 EDT; 2min 3s ago
  Invocation: 3848725aa9464966979d00f328f25f70

Sep 16 15:45:25 sykes-ubuntu systemd[1]: Reached target sleep.target - Sleep.
Sep 16 16:36:21 sykes-ubuntu systemd[1]: Stopped target sleep.target - Sleep.
bryansykes@sykes-ubuntu:~$

```

Figure 9 — All sleep targets masked — status shows "masked (Reason: Unit sleep.target is masked.)"

Result: sykes-ubuntu will never suspend or sleep under any circumstances. The machine remains online and SSH-accessible at all times regardless of lid position, inactivity period, or power management settings.

8. AZ-900 Exam Alignment

This project provides hands-on experience with several core AZ-900 (Microsoft Azure Fundamentals) exam domains. The table below maps each project activity to the corresponding exam objective:

AZ-900 Domain	Project Activity	Exam Relevance
Cloud Concepts	Using Azure Blob Storage for log backup. Understanding IaaS vs PaaS vs SaaS boundaries	Cloud storage models, shared responsibility
Azure Architecture	Creating resource groups, storage accounts in East US region	Resource groups, regions, subscriptions
Azure Storage	Provisioning Blob Storage, Standard LRS, creating containers, uploading blobs via CLI	Storage account types, redundancy options, blob tiers
Azure Identity & Access (RBAC)	Assigning Storage Blob Data Contributor role via Object ID at resource scope	RBAC, least privilege, role assignments
Azure Management Tools	Entire project executed via Azure CLI no portal GUI used for resource provisioning	Azure CLI, Azure Cloud Shell, management interfaces
Azure Security	HTTPS-only transfer enforced, server-side encryption confirmed, RBAC over account key access	Encryption, secure transfer, defense in depth

9. Key Lessons and Technical Notes

- Control plane vs data plane in Azure: Creating a storage account grants control plane access (management operations) but NOT data plane access (reading/writing blob data). The Storage Blob Data Contributor role must be explicitly assigned to write blobs even when you own the storage account. This is a common point of confusion and a real-world RBAC gotcha.
- Assigning RBAC by Object ID: Yahoo-federated Microsoft accounts cannot be resolved by email address in the Azure AD graph. The correct approach is `az ad signed-in-user show --query id -o tsv` to retrieve the Object ID and use `--assignee-object-id` in the role assignment command.

- Security defaults and device code flow: Microsoft's security defaults block device code authentication for certain account types. Disabling security defaults in Microsoft Entra ID (Azure AD → Properties → Manage Security Defaults) is required for CLI-based authentication with personal Microsoft accounts.
- Dynamic date variables in bash: `$(date +%Y-%m-%d)` creates a YYYY-MM-DD formatted date string at runtime. This pattern is used to name backup files uniquely per day, preventing overwrites and creating a natural archive structure in Blob Storage.
- `gzip -c` for non-destructive compression: The `-c` flag writes compressed output to stdout rather than replacing the original file. Piping this to a temp file (`> $BACKUP_FILE`) preserves the original `/var/log/syslog` while creating a compressed copy for upload.
- `2>&1` in cron output redirection: `>> backup.log` redirects stdout to the log file. `2>&1` additionally redirects stderr to the same destination. Without this, error output from failed runs would be silently discarded, making the cron job impossible to troubleshoot.
- Masking systemd targets vs disabling: `systemctl disable` removes a service from autostart but still allows manual invocation. `systemctl mask` creates a symlink to `/dev/null`, making the target completely uncalleable by any means, the correct choice when you want to permanently prevent sleep regardless of other settings.

10. Next Steps

Action	Purpose
Set up Azure Monitor / Log Analytics Workspace	Phase 3 of the project: monitor the upload pipeline, alert on failures, build dashboards
Add UFW log to backup scope	Include firewall logs alongside syslog for more comprehensive security-relevant data
Configure blob lifecycle management	Auto-delete logs older than 90 days to control storage costs
Add backup.log monitoring	Script to check backup.log daily and alert if upload failed closes the monitoring loop
Configure UFW firewall on sykes-ubuntu	Harden the machine, restrict to SSH port 22 only
Document in Azure Monitor	Connect sykes-ubuntu to a Log Analytics Workspace for centralized log visibility

11. Conclusion

This project successfully built an automated pipeline that compresses Ubuntu system logs on a physical home lab machine and uploads them to Microsoft Azure Blob Storage on a daily cron schedule all executed entirely via SSH from a Windows desktop workstation without any direct physical interaction with the Linux machine.

The project demonstrated practical proficiency in Azure CLI authentication and tenant management, resource group and storage account provisioning, RBAC role assignment at resource scope, bash scripting with dynamic variables and error handling, and cron job automation with output logging. Each of these skills maps directly to AZ-900 exam domains and to real-world cloud operations work in federal IT environments.

The troubleshooting challenges encountered. Microsoft security defaults, Yahoo account federation limitations, RBAC data plane vs control plane separation are not edge cases. They are the exact types of issues that appear in enterprise Azure environments and demonstrate the diagnostic thinking that separates hands-on practitioners from those who have only studied for certifications.

Documentation prepared by: Bryan Sykes | September 16, 2026 | Linux to Azure: Automated System Log Backup